Adaptive Timing Control for ThroughputLatency Trade-off in nFAPI Split

Ming-Hong HSU*, Ray-Guang Cheng*, and Co-author 1†
*Dept. of Electronic and Computer Engineering, National Taiwan University of Science and Technology, Taiwan
†Affiliation 2
Email: crg@mail.ntust.edu.tw

***Abstract*—nFAPI-based Split Option 6 separates the MAC scheduler and High-PHY across packet-switched transport, where scheduling messages must arrive at the PNF within a strict timing window. Because existing static slots ahead scheduling relies on a fixed lead time to trigger the scheduler, it cannot automatically adapt to varying network latency and jitter across different deployments. This paper proposes an adaptive timing control framework for nFAPI Split 6 that dynamically adjusts the scheduler trigger offset using PNF Timing Info feedback. The proposed method applies EWMA-based Timing Info and jitter estimation to increase the slots ahead value rapidly under timing risk and decrease it safely under stable conditions. We implement the framework in an OpenAirInterface-based end-to-end testbed with a commercial O-RU and COTS UE. Experimental results show that the proposed method maintains stable throughput under injected delay and jitter while avoiding unnecessary MAC HARQ RTT increase compared with static slots ahead baselines. Additional multi-switch/router deployment results further confirm that the proposed software-based method remains effective under realistic multi-hop transport conditions.**

***Index Terms*—nFAPI, O-RAN, Functional Split, Adaptive Control.**

I. Introduction

A. Background and Motivation

Functional split architectures have been widely studied as a key enabler for flexible and cost-efficient 5G/O-RAN deployments, where different protocol functions can be distributed across centralized and distributed nodes depending on bandwidth, latency, and coordination requirements [1], [2], [3]. In nFAPI-based Split Option 6, the MAC scheduler in the VNF must send scheduling control messages sufficiently ahead of the target slot so that the PNF can process them within its timing window. If a scheduling control message arrives later than the deadline, the PNF discards the scheduling request, resulting in missing-slot events and throughput degradation.

Existing OAI nFAPI-style implementations rely on statically configured slots ahead scheduling. Such a static configuration creates an inherent throughput-latency trade-off: a larger slots ahead value improves robustness against delay and jitter but increases MAC HARQ RTT, whereas a smaller value reduces latency but becomes vulnerable to late packet arrivals. Since HARQ retransmission timing is tightly coupled with MAC scheduling and resource availability, excessive scheduling lead time may inflate MAC-layer feedback latency and reduce the efficiency of HARQ process reuse [4]. When functional split interfaces are carried over non-ideal packet-switched fronthaul or midhaul networks, variable queueing delay and transport jitter may violate the strict timing constraints imposed by lower-layer processing [5], [6], [7].

However, existing open-source implementations such as OAI have historically relied on simplified `slot.indication` forwarding and static slots ahead configuration, leaving the SCF-defined Timing Info based delay management process incomplete. The SCF nFAPI specification defines software-level node sync and timing info procedures to support delay management between VNF and PNF entities, enabling timing compensation without relying solely on PNF-triggered `slot.indication` messages [8], [9].

To resolve this trade-off, this paper proposes an adaptive timing control framework for nFAPI Split 6. Our approach dynamically adjusts the scheduler trigger offset based on closed-loop feedback using PNF Timing Info reported by the PNF. By applying an Exponentially Weighted Moving Average (EWMA) model, the proposed method estimates the real-time Timing Info and transport jitter. The proposed method then applies a fast-increase and conservative-decrease policy to dynamically update the slots ahead, securing system stability under congestion while optimizing latency under stable network conditions.

This paper addresses the following problem: how to maintain nFAPI Split 6 scheduling stability over non-deterministic packet-switched transport without relying on hardware-level synchronization or statically configured worst-case lead times. The key challenge is that the scheduler must choose a slots ahead value before observing the future network delay. A conservative value improves robustness but increases MAC HARQ RTT, while an aggressive value reduces latency but risks late packet arrivals and slot loss. To resolve this trade-off, we propose a closed-loop delay management method that uses PNF Timing Info to estimate the Timing Info and adapt the slots ahead value online.

B. Related Works

Recent advancements in Open RAN (O-RAN) architectures have driven significant research into optimizing split-architecture performance and managing network latency. These works can be broadly classified into three categories.

Hardware-assisted Split 7.2: Hardware-assisted O-RAN testbeds such as X5G [10] demonstrate the feasibility of high-throughput end-to-end private 5G deployments by integrating OpenAirInterface with NVIDIA ARC, GPU acceleration, and high-performance networking support. However, such designs primarily improve performance through specialized acceleration and synchronization infrastructure, whereas this work targets a software-only timing adaptation method for nFAPI Split Option 6 over general-purpose packet-switched transport.

Intra-node Software Scheduling: Lee *et al.* [11] proposed an adaptive low-latency downlink transmission method for FAPI-based 5G NR gNBs, focusing on reducing intra-node processing delay within the MAC-PHY interaction. In contrast, our method addresses cross-node timing uncertainty caused by VNF-PNF transport delay and jitter in nFAPI Split Option 6. Similarly, Aquifer [12] optimized OS-level scheduling microservices to absorb processing delay spikes, but does not handle cross-node timing violations caused by unpredictable transport delay and jitter.

nFAPI Delay Management Gap: nFAPI has been proposed as a standard interface for 5G small cell networks [13], and open-source implementations of 5G NR Split Option 6 have demonstrated its feasibility in practical platforms [14]. Nevertheless, existing implementations still largely rely on static trigger offsets, leaving the SCF-defined Timing Info based delay-management loop underexplored. Our proposed EWMA-based adaptive slots ahead adjustment method fills this critical gap, providing a cost-effective, software-only solution for COTS servers.

Specifically, compared to Split 7.2 implementations like X5G [10] that require hardware-level PTP synchronization and GPU/SmartNIC offloads, or FAPI-based Split 6 designs [11] restricted to intra-node shared memory and process-level adjustments, our proposed method operates at the slot/TTI level on standard COTS servers, actively managing cross-node network jitter and delay over standard packet-switched Ethernet without hardware acceleration dependencies.

C. Research Scope and Contributions

In this paper, we focus on the nFAPI-based Option 6 split as a representative case of timing-sensitive yet transport-flexible RAN disaggregation. While Option 6 offers significant advantages in terms of reduced fronthaul bandwidth and deployment flexibility, its performance is highly sensitive to non-deterministic latency introduced by transport networks and server processing.

To address the limitations of static configurations under such non-deterministic transport environments, this paper introduces a dynamic timing adjustment framework that achieves closed-loop synchronization stability. The main contributions of this paper are summarized as follows:

- **SCF-compliant nFAPI timing framework for OAI Split 6:** We redesign the OAI nFAPI timing path to support Node Sync, PNF Timing Info, and Delay Management, enabling closed-loop timing control instead of simplified static slot forwarding.
- **EWMA-based adaptive slots ahead adjustment:** We propose a lightweight closed-loop delay-management method that estimates the mean Timing Info and jitter from PNF Timing Info and dynamically adjusts the scheduler trigger offset to balance throughput stability and MAC HARQ RTT.
- **End-to-end validation under controlled and multi-hop transport:** We implement the proposed method on an OpenAirInterface-based E2E physical testbed with a commercial Pegatron O-RU and COTS UE. We evaluate throughput, MAC HARQ RTT, missing-slot behavior, and stability under both Linux-TC-injected delay/jitter (under Experimental Scenario I) and a realistic 3-switch/3-router campus network deployment (under Experimental Scenario III). The implementation has been successfully merged into the official upstream OAI codebase to support reproducibility and future research.

II. System Model

Before diving into the system details, this study introduces the adopted system framework. To integrate the Network Functional Application Platform Interface (nFAPI) with mainstream commercial equipment, the proposed system adopts a stable two-stage split architecture. The network first connects via the Split 6 nFAPI interface between the network functions, and then translates to the standard O-RAN Split 7.2x interface to connect to a commercial Pegatron O-RU. Figure 1 illustrates this proposed system architecture and its delay management mechanism.

A. End-to-End Architecture

The system establishes a complete 5G end-to-end communication link, extending from the core to the user equipment:

- **Core Network (CN):** The central element responsible for routing, session management, and authenticating user data.
- **O-RAN Central Unit (O-CU):** A logical node that handles higher-layer protocol stacks, such as the Radio Resource Control (RRC) and Packet Data Convergence Protocol (PDCP). It connects with distributed units via the F1 interface.
- **O-RAN Distributed Unit (O-DU):** To optimize computational performance and deployment flexibility, the system separates the O-DU into an O-DU High and an O-DU Low. The O-DU High is executed as a Virtualized Network Function (VNF) on a virtualized machine, whereas the O-DU Low is deployed as a Physical Network Function (PNF) on dedicated hardware.
- **Switch:** A commercial network switch is located between the VNF and PNF. This mid-layer connection utilizes the nFAPI protocol. However, traversing this packet-switched network inevitably generates non-deterministic latency.
- **O-RAN Radio Unit (O-RU):** A specialized hardware unit responsible for low-level physical layer (Low PHY) processing, digital beamforming, and Radio Frequency (RF) transmission.

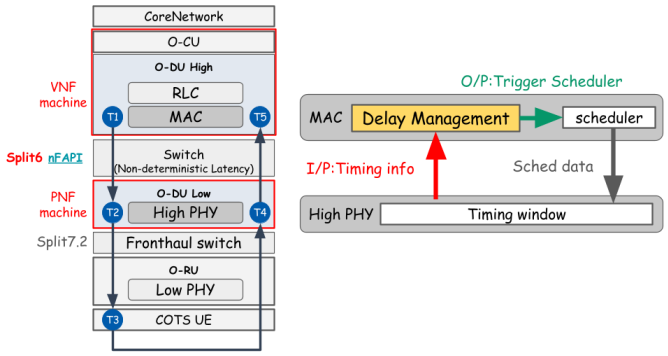


Fig. 1. System architecture and delay management mechanism.

- **Commercial Off-The-Shelf User Equipment (COTS UE):** Standard commercial smartphones or modems that act as the terminal receivers in the network.

B. O-DU High: MAC, Scheduler, and Timing Core

Within the O-DU High virtualized network function, the timing control is organized as a unified timing control system. The core processes handling this timing core include the Scheduler, Node sync, and Delay Management:

- **Scheduler:** Critical for allocating the available radio resources (such as time-frequency resource blocks) and determining the exact priority and timing of downlink/uplink data transmissions.
- **Node sync:** Responsible for software-level clock synchronization. This procedure periodically exchanges synchronization timestamps ($t_1 \sim t_4$) with the PNF via standard *DL/UL_node_sync* messages. It calculates the instantaneous clock offset and applies slot-level (S_{adj}) and microsecond-level (μs_{adj}) adjustments to align the VNF's local clock to the PNF baseline, eliminating long-term hardware clock drift.
- **Delay Management:** Operates on top of the stabilized clock phase established by Node sync. This mechanism utilizes the PNF Timing Info ($TimingInfo[i]$) feedback from the PNF to dynamically estimate transport network jitter. It then actively adjusts the slots ahead (S_{ahead}) trigger offset of the scheduler. Here, *slots ahead* (or *VNF slots ahead*) refers to the number of slots by which the VNF triggers the scheduler in advance relative to the reference timing info. This control compensates for the non-deterministic packet transport delays and VNF processing delays without inducing synchronization loss.

C. O-DU Low: High PHY and Timing Window

The O-DU Low manages the higher-level baseband processing of the physical layer (High PHY). At this level, the most critical synchronization mechanism is the nFAPI PNF Timing Window:

- **Timing Window:** A predefined, strict time duration. Let T_{window} denote the duration of this PNF timing window, and T_{slot} denote the subframe slot duration. When the scheduling message sent by the O-DU High propagates across the network, it must accurately arrive and fall exactly within this Window. Otherwise, the High PHY will reject the message and fail to process the slot data.
- **Timing Info ($TimingInfo[i]$):** The PNF continuously monitors the exact timing info of control messages relative to the transmission deadline. This information, denoted as $TimingInfo[i]$, represents the remaining time margin within the timing window. The PNF periodically sends this information back to the MAC layer for Delay Management. This establishes a closed-loop feedback control system, compensating for the unpredictable transmission latency variations between the VNF and PNF machines.

D. Key Validation Metrics

To evaluate the stability, synchronization accuracy, and real-time performance of the proposed algorithm under non-ideal environments, this study defines the following Key Validation Metrics:

- **MAC Layer HARQ RTT ($T_5 - T_1$):** Measures the Hybrid Automatic Repeat Request (HARQ) loop delay specifically at the MAC layer. This metric accurately reflects the feedback efficiency and operational health of the link layer.
- **VNF-PNF DL Latency ($T_2 - T_1$):** Measures the one-way downlink latency accumulated from the virtualized machine (VNF) to the physical machine (PNF). This metric is used to directly quantify and analyze the delay jitter impact injected by the intermediate Ethernet switch.

Note that RTT denotes the Round-Trip Time. To validate the split-architecture stability under non-ideal fronthaul conditions, network latency simulation is conducted at the uplink ($T_5 - T_4$) and downlink ($T_2 - T_1$) segments via Linux Traffic Control (TC).

To validate the proposed performance optimization within a realistic network environment, the system design leverages a commercial-grade deployment framework:

- **O-RAN Split 7.2x Interoperability:** While the Small Cell Forum (SCF) has standardized the nFAPI for the MAC-PHY split (Option 6) [13], commercial-grade Open Radio Units (O-RUs) natively support the O-RAN Split Option 7.2x. To achieve seamless integration with open-source stacks like OpenAirInterface [14], our system bridges these interfaces.
- **Hybrid System Architecture for E2E Validation:** To validate the nFAPI performance over a fully commercial ecosystem, this paper adopts a hybrid architecture (Split Option 6 + Split Option 7.2). This setup establishes a translation/interworking layer that enables the nFAPI-driven MAC/High-PHY to successfully interact with a commercial Pegatron O-RU, facilitating end-to-end verification using Commercial Off-The-Shelf (COTS) UEs.

III. Proposed Method

This section details the proposed dynamic timing adjustment framework. To resolve the unstable latency during packet transmission between the PNF and VNF, we propose two core timing control algorithms: Node sync for software-level clock synchronization and Delay Management for slot-level adaptive delay management. The timing control of this system is organized as a closed-loop system, ensuring that the VNF-side scheduling can dynamically track and compensate for both clock drift and slot-level transport latency variation. The two algorithms cooperate as follows:

1) **Node sync:** Maintains phase and slot boundary alignment between the VNF and PNF. It periodically exchanges timestamps to calculate the absolute phase offset, correcting for hardware clock drift.
2) **Delay Management:** Tracks and estimates packet transport delays and jitter at the slot level, dynamically adjusting the slots ahead (S_{ahead}) to guarantee that control messages fall within the PNF's strict timing window.

A. Node sync

To perform software-level clock synchronization, the system adopts a timestamp exchange mechanism. The VNF periodically transmits a downlink node synchronization message with a local timestamp t_1. Upon receiving the message, the PNF records the local arrival timestamp t_2 and transmits an uplink node synchronization message back to the VNF, including the transmit timestamp t_3 and the echoed t_2. When the VNF receives the message, it records its local arrival timestamp t_4. Because the nFAPI timestamps are constrained by a 1024 subframe number (SFN) loop, they wrap around every 10.24 seconds (10240000 µs). The algorithm calculates the differences $d_1 = t_2 - t_1$ and $d_2 = t_4 - t_3$, applying a wrap-around correction to align them within the interval $[-5120000, 5120000]$ µs. The VNF then calculates the instantaneous clock offset:

$$offset = \frac{d_1 - d_2}{2} \tag{1}$$

To prevent timing adjustments from causing control loop oscillations, the synchronization control loop implements a state-based locking mechanism. The system has two synchronization states: locked ($sync\_locked = 1$) and unlocked ($sync\_locked = 0$). In the locked state, the VNF monitors the drift of the local clock. If the absolute offset exceeds the slot duration T_{slot}:

$$|offset| > T_{\text{slot}} \tag{2}$$

the VNF unlocks the synchronization ($sync\_locked = 0$) to trigger re-synchronization. In the unlocked state, the VNF checks whether the offset has converged within the slot duration T_{slot}. If $|offset| \leq T_{\text{slot}}$, the system locks the synchronization state ($sync\_locked = 1$). Otherwise, the VNF decomposes the offset into a slot-level adjustment S_{adj} and a microsecond-level adjustment μs_{adj} based on the slot duration T_{slot}:

$$S_{\text{adj}} = offset / T_{\text{slot}} \tag{3}$$

$$\mu s_{\text{adj}} = offset \pmod{T_{\text{slot}}} \tag{4}$$

These adjustments are accumulated into the global adjustment variables:

$$slot\_adjustment \leftarrow slot\_adjustment + S_{\text{adj}} \tag{5}$$

$$us\_adjustment \leftarrow us\_adjustment - \mu s_{\text{adj}} \tag{6}$$

The negative sign in the microsecond adjustment reduces the local thread sleep time when the VNF clock is behind, speeding up the slot transition rate to catch up. The synchronization process is summarized in Algorithm 1.

Algorithm 1 Node sync

Input: t_1, t_2, t_3 (Timestamps from PNF *UL_node_sync*), t_4 (VNF local receive timestamp)

Output: *slot_adjustment* (Slot adjustment accumulator), *us_adjustment* (Microsecond adjustment accumulator)

Notation:

offset (Instantaneous timing offset), *sync_locked* (Synchronization lock state)

T_{slot} (Slot duration in µs), T_{wrap} (Wrap-around period, 10240000 µs)

WrapCorrection(x, T) (Corrects x within $[-T/2, T/2]$)

Phase I: Timing Offset Calculation

$d_1 \leftarrow \text{WrapCorrection}(t_2 - t_1, T_{\text{wrap}})$
$d_2 \leftarrow \text{WrapCorrection}(t_4 - t_3, T_{\text{wrap}})$
$offset \leftarrow (d_1 - d_2)/2$

Phase II: Hysteresis Lock and Clock Adjustment

if $sync\_locked = 1 \wedge |offset| > T_{\text{slot}}$ **then**
    $sync\_locked \leftarrow 0$ ▷ Unlock due to excessive clock drift
end if
if $sync\_locked = 0$ **then**
    if $|offset| \leq T_{\text{slot}}$ **then**
        $sync\_locked \leftarrow 1$ ▷ Lock synchronization
    else
        $S_{\text{adj}} \leftarrow offset / T_{\text{slot}}$ ▷ Integer division
        $\mu s_{\text{adj}} \leftarrow offset \pmod{T_{\text{slot}}}$
        $slot\_adjustment \leftarrow slot\_adjustment + S_{\text{adj}}$
        $us\_adjustment \leftarrow us\_adjustment - \mu s_{\text{adj}}$
    end if
end if
return $slot\_adjustment, us\_adjustment$

B. Delay Management

To counteract changing network latency and maintain scheduling stability, a dynamic delay-management strategy is implemented. This strategy operates as a closed-loop feedback mechanism based on standard Small Cell Forum (SCF) PNF Timing Info reported by the PNF. By evaluating the Timing Info relative to the PNF deadline, the proposed method dynamically adjusts the scheduling lead time S_{ahead}.

1) Timing Model: Let T_{slot} denote the slot duration and S_{ahead} denote the number of slots by which the VNF scheduler triggers earlier than the target slot. For a scheduling message generated at VNF time t_g, the corresponding target PNF processing deadline is determined by the target slot boundary. The Timing Info reported by the PNF is denoted by $TimingInfo[i]$, where $TimingInfo[i] < 0$ indicates that the message arrives before the deadline, and $TimingInfo[i] > 0$ indicates a late arrival. Therefore, the delay-management objective is to select the minimum stable S_{ahead} such that late arrivals are avoided ($TimingInfo[i] \leq 0$) while MAC HARQ RTT is not unnecessarily inflated.

2) EWMA-based Timing-Info Estimation: To smooth high-frequency network noise while remaining sensitive to timing drift, EWMA is used to smooth the $TimingInfo[i]$ and to compute $TimingInfoDev[i]$ and $TimingInfoEWMA[i]$. The EWMA structure is inspired by the classical TCP round-trip time estimation method, where a smoothed estimate and a deviation estimate are jointly maintained to tolerate delay variation while remaining responsive to congestion-induced timing changes [15], [16]. The EWMA updates for mean Timing Info and jitter are defined as:

$$\begin{aligned} &TimingInfoEWMA[i] \\ &\quad = (1-\alpha)\, TimingInfoEWMA[i-1] \\ &\quad\quad + \alpha\, TimingInfo[i] \end{aligned} \tag{7}$$

$$\begin{aligned} &TimingInfoDev[i] \\ &\quad = (1-\beta)\, TimingInfoDev[i-1] \\ &\quad\quad + \beta |\, TimingInfo[i] - TimingInfoEWMA[i]| \end{aligned} \tag{8}$$

Following this dual-gain estimation principle, the smoothing factors are set to $\alpha = 1/8$ for long-term Timing-Info tracking and $\beta = 1/4$ for fast jitter-variation estimation. Both parameters are negative powers of two ($\alpha = 2^{-3}$, $\beta = 2^{-2}$), which allows efficient fixed-point implementation using bitwise shift operations in the timing-critical MAC path.

The combined timing risk indicator R_i is evaluated to determine whether the Timing Info crosses the deadline boundary:

$$R_i = TimingInfoEWMA[i] + TimingInfoDev[i] \tag{9}$$

A positive timing risk ($R_i > 0$) implies that the estimated worst-case Timing Info crosses the deadline, indicating an anomaly condition, whereas a negative risk indicates a safe margin.

3) Adaptive Slots Ahead Control: The update of the slots ahead trigger timing S_{ahead} is governed by a fast-increase and conservative-decrease control law to optimize latency while preserving throughput under dynamic congestion:

- **Rapid Increase (Preserve Throughput):** If an anomaly timing risk is detected ($R_i > 0$), indicating that the scheduling message is crossing the PNF processing deadline, the proposed method immediately increases the slots ahead value to prevent late packet arrivals and missing-slot dropouts:

$$S_{\text{ahead},i+1} = \min\left(S_{\text{ahead},i} + \left\lceil \frac{R_i}{T_{\text{slot}}} \right\rceil, S_{\max}\right) \tag{10}$$

  and the stable observation counter C_{stable} is reset to 0. Here, $S_{\max} = \lfloor T_{\text{window}}/T_{\text{slot}} \rfloor$ represents the maximum allowed slots ahead.
- **Gradual Decrease (Optimize Latency):** If the system is under stable and safe conditions ($R_i < -TimingInfoDev[i]$), indicating that there is a sufficient timing margin, the proposed method decrements the slots ahead value by one slot once the stability condition is satisfied ($C_{\text{stable}} \geq C_{\text{threshold}}$):

$$S_{\text{ahead},i+1} = \max\left(S_{\text{ahead},i} - 1, S_{\min}\right) \tag{11}$$

  and resets $C_{\text{stable}} = 0$ to prevent premature timing reductions. Here, $S_{\min} = 1$ is the minimum slots ahead trigger offset.
- **Maintain Baseline:** Otherwise, the trigger timing remains unchanged:

$$S_{\text{ahead},i+1} = S_{\text{ahead},i} \tag{12}$$

  and the stable counter C_{stable} is incremented if the current sample is safe ($R_i \leq 0$).

The core delay management strategy is condensed in Algorithm 2.

C. Coordination between Node sync and Delay Management

To ensure stable operation under real-world conditions, node sync and delay management must coordinate without inducing control loop oscillations. The microsecond-level node sync focuses strictly on correcting clock phase offsets to align the VNF's scheduling thread with the PNF's slot transitions. Because it directly shifts the VNF's time base, any clock adjustment ($S_{adj}, \mu s_{adj}$) temporarily shifts the arrival timestamp reference. Delay management is designed to absorb this transient shifting. By relying on history-weighted EWMA-based risk tracking (R_i), delay management avoids reacting to the temporary timing offsets caused by clock adjustments, allowing node sync to converge. Once node sync locks synchronization ($sync\_locked = 1$), the time base stabilizes, allowing delay management to achieve precise slot-level optimizations (S_{ahead}).

IV. Experimental Results

Experiments were conducted to validate the proposed method in an OpenAirInterface (OAI) nFAPI end-to-end environment. OpenAirInterface has been widely used as an open-source platform for 5G NR end-to-end experimentation and system-level validation [17], [14]. Each reported throughput value represents the average of 180 data points, collected from three independent trials with 60 samples per trial. The evaluation focuses on six aspects of the proposed method. First, we examine whether EWMA-based filtering can stabilize noisy PNF timing info feedback under transport network jitter. Second, we compare the proposed adaptive slots ahead adjustment with static baselines to evaluate its ability to maintain throughput while avoiding unnecessary MAC HARQ RTT increase. Third, we evaluate the closed-loop delay management method under severe congestion and dynamic

Algorithm 2 Delay Management

Input: $TimingInfo[i]$ (Observed PNF Timing Info), S_{curr} (Current trigger timing), T_{window} (Timing window), T_{slot} (Slot duration)

Output: S_{next} (Trigger timing for the next MAC schedule)

Notation:
$TimingInfoEWMA$ (smoothed mean PNF Timing Info), $TimingInfoDev$ (smoothed arrival jitter)
R (Timing risk indicator), C_{stable} (Stable observation counter)
$C_{\text{threshold}}$ (Stability observation threshold), $S_{\max}, S_{\min}$ (Boundary limits)
α, β (Smoothing factors for latency and jitter)

Phase I: Latency and Jitter Estimation

$TimingInfoEWMA \leftarrow (1-\alpha)\,TimingInfoEWMA + \alpha\,TimingInfo[i]$ ▷ Update mean PNF Timing Info

$TimingInfoDev \leftarrow (1-\beta)\,TimingInfoDev + \beta|TimingInfo[i] - TimingInfoEWMA|$ ▷ Update arrival jitter

$R \leftarrow TimingInfoEWMA + TimingInfoDev$ ▷ Evaluate combined risk indicator

Phase II: Fast-Up and Slow-Down Timing Adjustment

if $R > 0$ **then**
  $S_{\text{next}} \leftarrow \min(S_{\text{curr}} + \lceil R/T_{\text{slot}} \rceil, S_{\max})$ ▷ Rapid Increase
  $C_{\text{stable}} \leftarrow 0$
else if $R < -TimingInfoDev$ **then**
  if $C_{\text{stable}} \geq C_{\text{threshold}}$ **then**
    $S_{\text{next}} \leftarrow \max(S_{\text{curr}} - 1, S_{\min})$ ▷ Gradual Decrease
    $C_{\text{stable}} \leftarrow 0$
  else
    $S_{\text{next}} \leftarrow S_{\text{curr}}$
    $C_{\text{stable}} \leftarrow C_{\text{stable}} + 1$
  end if
else
  $S_{\text{next}} \leftarrow S_{\text{curr}}$
  if $R \leq 0$ **then**
    $C_{\text{stable}} \leftarrow C_{\text{stable}} + 1$
  else
    $C_{\text{stable}} \leftarrow 0$
  end if
end if
return S_{next}

jitter to verify end-to-end synchronization stability. Fourth, we evaluate the synchronization offset of the node sync algorithm under a multi-hop transport path. Fifth, we analyze the stability under real network congestion in a realistic campus network deployment. Finally, we evaluate the impact of different timing info reporting modes and periods under varying deployment distances.

The rest of this section is organized as follows. Section IV-A describes the hardware, software, and automated rApp evaluation framework, including Experimental Scenario I, Experimental Scenario II, and Experimental Scenario III. Section IV-B details the system-level integration and software validation. Section IV-C (Experimental 1) validates EWMA-based PNF timing info stabilization. Section IV-D (Experimental 2) compares throughput and MAC HARQ RTT against static slots ahead baselines. Section IV-E (Experimental 3) evaluates robustness under injected delay and jitter. Section IV-F (Experimental 4) evaluates the synchronization offset under the multi-hop deployment. Section IV-G (Experimental 5) presents the stability under real network congestion in the campus network. Finally, Section IV-H (Experimental 6) evaluates the performance under different timing info reporting modes and periods.

A. Experimental Scenario and rApp Framework

To evaluate the performance of our proposed timing adjustment framework under different environments, we design three topologies:

- **Experimental Scenario I (Controlled Single-Switch environment):** A standard testbed where the VNF and PNF are connected directly via a single commercial Ethernet switch. This controlled setup is used to analyze EWMA filtering, benchmark throughput-latency trade-offs against static baselines, and evaluate robustness under Linux-TC-injected delay and jitter.
- **Experimental Scenario II (Near-Site Single-Hop Deployment):** A near-site campus deployment where the VNF and PNF are co-located in the same campus area and connected via a single commercial Ethernet switch, introducing minimal base transport latency (around 0.3–0.5 ms). This scenario evaluates the proposed timing control under stable, low-latency fronthaul conditions.
- **Experimental Scenario III (Multi-Switch/Router Campus Deployment):** A realistic multi-hop campus network topology where the VNF and PNF are physically disaggregated across different campus buildings. As shown in Fig. 3, the connection traverses 3 commercial switches and 3 commercial routers, introducing realistic transport delays and route-dependent jitter (ping latency up to 2.0 ms).

The detailed software and hardware configurations for the end-to-end network deployment in the testbed environment are outlined below.

To ensure automated measurement and analysis with consistent data quality, an automated evaluation rApp was developed

TABLE I
TESTBED NODE CONFIGURATION

Unit	Specification
Core Network (CN)	Open5GS
SW Stack / Branch	OpenAirInterface (`2026.w13`)
VNF Server	Intel® Xeon® Gold 6226R
PNF Server	Intel® Xeon® Gold 6433N
O-RU	Pegatron RU (P5G-Indoor O-RU-PR1450)
COTS UE	Samsung Galaxy S20

TABLE II
WIRELESS SYSTEM PARAMETERS

Parameter	Setting
Subcarrier Spacing	30 kHz
TDD Pattern	3D1S1U
MIMO / Ports	4-layer / 4T4R
Fronthaul	O-RAN F release

to operate on the Non-Real-Time RAN Intelligent Controller (Non-RT RIC) within the O-RAN Service Management and Orchestration (SMO) framework [18]. As shown in Fig. 2, this rApp automates the start-up of the VNF and PNF. Once the gNB is established, it automatically connects to the control PC via SSH to toggle the UE's airplane mode using Android Debug Bridge (ADB), and subsequently uses SSH to launch the iPerf server on the Core Network (CN) server. It then executes a series of pre-scheduled iPerf tests automatically. Finally, the rApp collects all log files via O1 SFTP and utilizes Python scripts to plot the following data analysis charts. Crucially, it manages the network latency simulation at $T_5 - T_4$ and $T_2 - T_1$ via Linux Traffic Control (TC) to systematically validate the split-architecture stability.

B. Thorough System-Level Integration and Validation

To prove the stability, practical deployability, and compatibility of the proposed timing control and nFAPI modifications, our software implementation was submitted and successfully merged into the official upstream OpenAirInterface (OAI) repository. Beyond our localized end-to-end evaluation using the rApp framework, the proposed solution went through thorough testing in the official OAI Continuous Integration and Continuous Deployment (CI/CD) pipeline. Specifically, the code successfully passed all image building and cross-compilation tests across standard x86 and ARM64-based architectures, including enterprise-grade Red Hat Enterprise Linux (RHEL) clusters and NVIDIA Jetson edge platforms. Furthermore, our implementation successfully completed key system-level integration and physical layer regression tests. These tests include 5G Standalone (SA) end-to-end integration, real radio frequency (RF) transmissions using USRP B200 and AW2S radio units, standard F1/N2 interface control-plane handover procedures, and 3GPP-compliant physical layer channel simulations. These regression and compilation results confirm that our proposed software-defined timing loops are highly robust, backward-compatible, and ready for

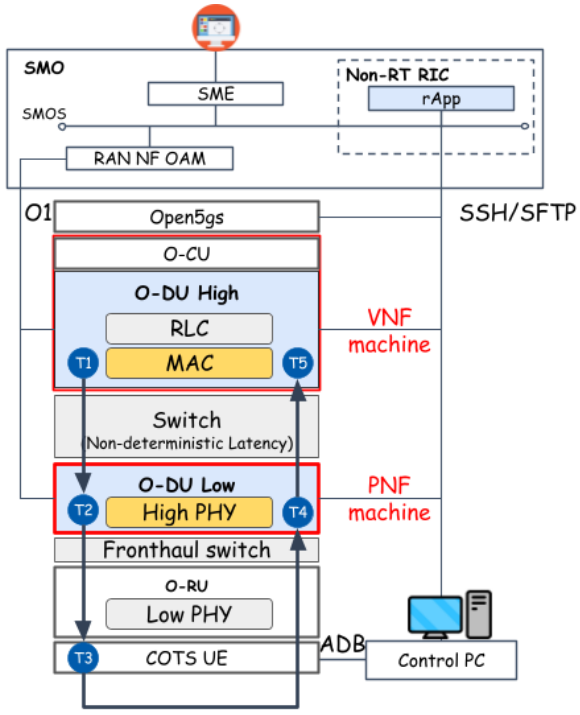


Fig. 2. Architecture of the rApp automated evaluation framework.

diverse commercial cloud hardware deployments without introducing any hardware dependency.

C. Experimental 1: Validation of EWMA for PNF Timing Info

This scenario validates the application of an Exponentially Weighted Moving Average (EWMA) to smooth the PNF Timing Info ($TimingInfo[i]$). Fig. 4(a) shows the timing control directly using the raw PNF Timing Info ($TimingInfo[i]$), where the raw timing values change a lot due to network jitter. Fig. 4(b) is the cumulative count of the packets exceeding the deadline when the timing adjustment directly uses the raw PNF Timing Info ($TimingInfo[i]$). The EWMA mechanism filters out the high-frequency jitter, as shown in the PNF Timing Info ($TimingInfo[i]$) after applying EWMA in Fig. 4(c). Finally, Fig. 4(d) shows the cumulative count of the packets exceeding the deadline under the EWMA-based timing control. The results show that applying EWMA allows the timing loop to adjust the slots ahead stably, which significantly reduces the number of packets that arrive after the deadline.

D. Experimental 2: Performance: Throughput & MAC HARQ RTT

We benchmark the peak throughput and MAC HARQ RTT against a static baseline. Under stable fronthaul conditions, as shown in Fig. 5, the proposed algorithm prioritizes guaranteeing the optimal throughput under different offered loads, consistently maintaining near-peak throughput performance. Simultaneously, while ensuring this optimal throughput, the algorithm also optimizes the selection of the slots ahead

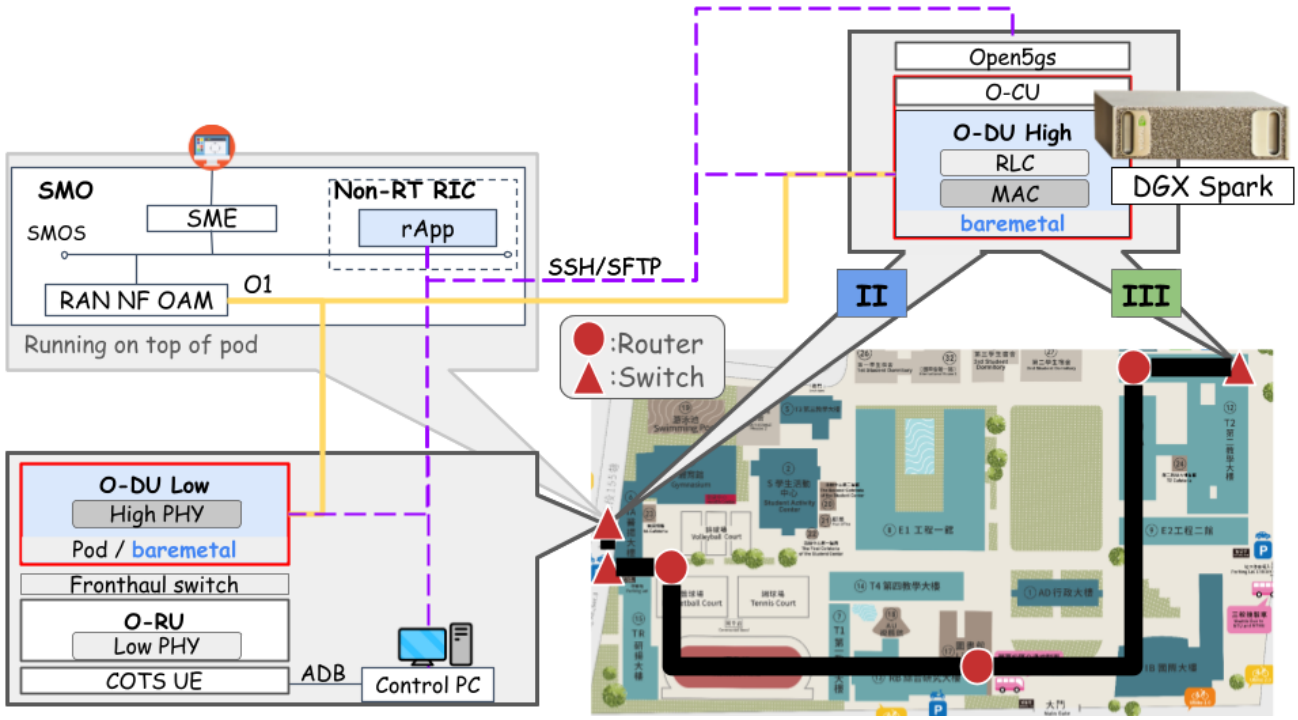
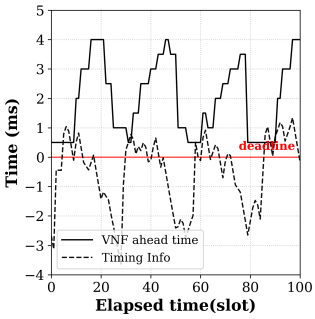
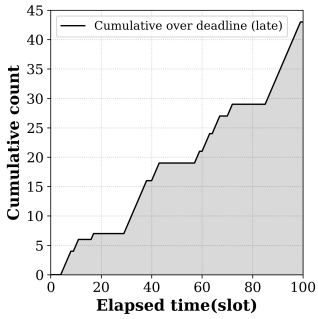


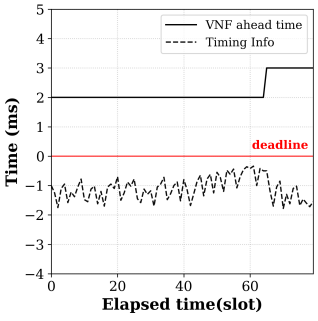
Fig. 3. Experimental Scenario III: multi-switch/router campus deployment for evaluating the proposed adaptive timing control under realistic non-deterministic transport.



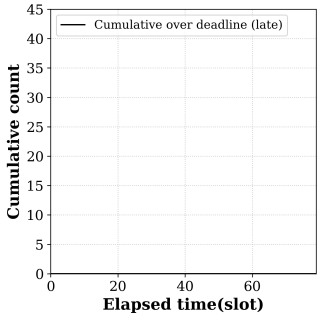
(a) Direct Use of $TimingInfo[i]$



(b) Cumulative Count of Packets Exceeding Deadline under Direct Use



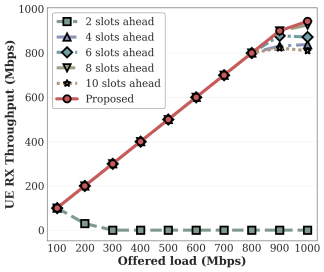
(c) EWMA Application on $TimingInfo[i]$



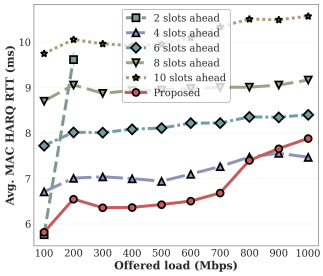
(d) Cumulative Count of Packets Exceeding Deadline under EWMA

Fig. 4. Validation of EWMA processing on PNF Timing Info.

trigger offset to optimize the MAC layer HARQ Round-Trip Time (RTT), effectively avoiding unnecessary overhead. Specifically, at 1000 Mbps offered load, the proposed method achieves approximately 940 Mbps throughput with a MAC HARQ RTT of about 8 mscomparable to the 4-slot-ahead RTTwhile the static 10-slot-ahead configuration incurs approximately 10 ms RTT with no throughput advantage. The 2-slot-ahead configuration collapses to near-zero throughput beyond 200 Mbps offered load.



(a) Throughput Optimization



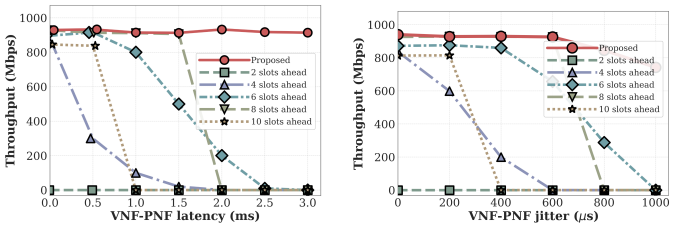
(b) MAC Latency & Reliability

Fig. 5. Performance evaluation: MAC RTT efficiency and system throughput.

E. Experimental 3: Stability under Traffic Control Network Congestion

To evaluate stability, we inject delay and jitter between the VNF and PNF using Linux TC. As shown in Fig. 6a, static slots ahead configurations suffer from synchronization

failures and throughput drops when delay exceeds the timing window, whereas our proposed adaptive mechanism dynamically increases the slots ahead to maintain stable throughput. Specifically, the proposed method maintains approximately 930 Mbps across all tested VNF-PNF latency values up to 3.0 ms, while the best-performing static configuration (10 slots ahead) degrades to near-zero throughput at approximately 2.5 ms injected latency. Additionally, we evaluate the UE DL RX throughput under a saturated 1 Gbps UDP offered load with unpredictable link jitter. As shown in Fig. 6b, while fixed slots ahead configurations experience severe throughput degradation due to late packets, the proposed adaptive method dynamically tracks jitter and secures stable, high throughput at the UE. The proposed method sustains near-peak throughput (approximately 930 Mbps) across the full tested jitter range up to 1000 μs, while static 8-slot-ahead configurations drop sharply beyond approximately 600 μs.



(a) Stability under Delay Congestion (b) Throughput under Unpredictable Jitter

Fig. 6. System stability and throughput performance under varying network conditions.

F. Experimental 4: Synchronization Offset

To evaluate the synchronization accuracy of the proposed node sync algorithm under a multi-hop transport network, we deploy the VNF and PNF across the campus network in Experimental Scenario III. As shown in Fig. 3, the VNF-side components are placed at a remote computing site, while the radio-side components and UE remain at the local radio site. Fig. 7 shows the measured VNF-PNF synchronization offset in microseconds over elapsed slots. When the VNF and PNF establish their connection, they start calculating slot 0 independently. Program start-up timing differences cause an initial program execution timing offset. Additionally, their clock phase drifts during operation. When the accumulated offset exceeds one slot (500 μs for subcarrier spacing of 30 kHz and slot duration of 500 μs), the system triggers the node sync algorithm to adjust the VNF's scheduling time base. This aligns the VNF's slot boundaries with the PNF, correcting the time-base offset and stabilizing the synchronization offset between 0 and 100 μs.

G. Experimental 5: Stability under Real Network Congestion

We evaluate the throughput and ping latency under different real network deployment topologies. We use ping to measure the network interface latency between the VNF and PNF. Under Scenario II (Near-site deployment, Fig. 8(a)), the base

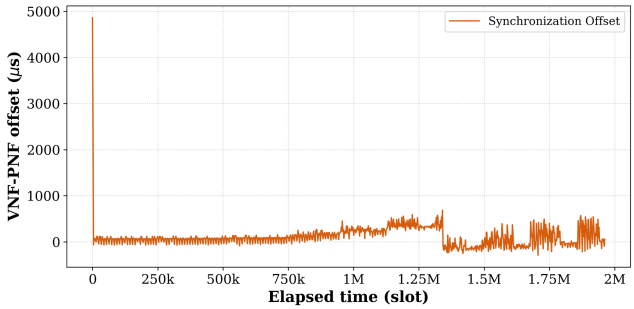
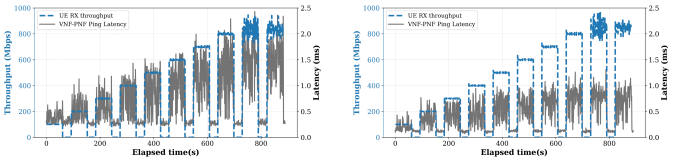


Fig. 7. VNF-PNF synchronization offset over elapsed slots under the multi-switch/router deployment.

latency is lower and more stable (around 0.3 to 0.5 ms). The proposed method maintains stable near-peak throughput, and because the transport delay is lower, the required slots ahead parameter is smaller, which also optimizes the MAC HARQ RTT. Under Scenario III (Remote deployment, Fig. 8(b)), the connection spans the campus network and traverses multiple switches and routers, resulting in higher base latency and queueing delay variations (ping latency up to 2.0 ms). Despite this severe transport jitter, the proposed method dynamically adapts the slots ahead parameter to prevent late packet arrivals, maintaining stable data transmission.

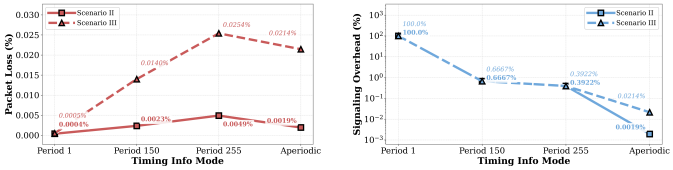


(a) Scenario II: Near-site deployment (b) Scenario III: Remote deployment

Fig. 8. UE RX throughput and ping latency under different real network deployment scenarios.

H. Experimental 6: Timing Info Mode

We evaluate the impact of different timing info reporting modes and periods under Scenario II (Near-site deployment, Fig. 9(a)) and Scenario III (Remote deployment, Fig. 9(b)). The nFAPI protocol supports periodic and aperiodic timing info feedback modes. Fig. 9(a) and Fig. 9(b) present the packet loss rate under different reporting configurations for both deployment scenarios. In both cases, increasing the timing period in periodic mode leads to a higher packet loss rate. This is because a longer period reduces feedback frequency. As a result, the VNF timing loop cannot adjust the slots ahead parameter fast enough to adapt to network variations, causing more slot packets to arrive late and be discarded by the PNF. Under Scenario III (Remote deployment), the longer distance and higher transport jitter result in a higher base packet loss rate, making the system more sensitive to feedback delay. Under Scenario II (Near-site deployment), the lower base latency and jitter allow the system to maintain a lower packet loss rate even with a slightly longer timing period.



(a) Scenario II: Near-site deployment (b) Scenario III: Remote deployment

Fig. 9. Evaluation of packet loss rate under different deployment scenarios.

V. Conclusion

This paper presented a dynamic timing adjustment framework for disaggregated RAN architectures, focusing on nFAPI-based Split Option 6 over non-ideal packet-switched networks. To address the limitations of static slots ahead configurations under unpredictable network jitter and server processing delay, the proposed method applies EWMA-based arrival-margin and jitter estimation to dynamically adjust the scheduler trigger timing. Experimental validation on the OpenAirInterface platform with a commercial Pegatron O-RU shows that the proposed method reduces synchronization failures and maintains stable throughput under severe delay and jitter conditions. Compared with hardware-centric approaches that rely on specialized acceleration or precise synchronization infrastructure, the proposed software-based method provides a cost-effective and flexible alternative for robust O-RAN deployment on general-purpose COTS servers.

Although the proposed software-based timing control achieves robust performance, the current evaluation focuses primarily on downlink timing control under controlled delay and jitter injection. Future work will extend the proposed method to joint uplink/downlink timing optimization, multi-cell deployments, and analytical stability bounds for the feedback loop.

References

[1] L. M. P. Larsen, A. Checko, and H. L. Christiansen, "A survey of the functional splits proposed for 5g mobile crosshaul networks," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 146–172, 2019.

[2] *TR 38.801 Study on new radio access technology: Radio access architecture and interfaces*, 3rd Generation Partnership Project (3GPP), 2017, version 14.0.0. [Online]. Available: https://portal.3gpp.org/desktopmodules/Specifications/SpecificationDetails.aspx?specificationId=3066

[3] K. Alam, M. A. Habibi, M. Tammen, D. Krummacker, W. Saad, M. Di Renzo, T. Melodia, X. Costa-Pérez, M. Debbah, A. Dutta, and H. D. Schotten, "A comprehensive tutorial and survey of O-RAN: Exploring slicing-aware architecture, deployment options, use cases, and challenges," *IEEE Communications Surveys & Tutorials*, vol. 28, pp. 1637–1678, 2026.

[4] M. Elfiky, Z. Becvar, and P. Mach, "Allocation of resources for HARQ retransmission in mobile networks based on C-RAN," *IEEE Systems Journal*, vol. 18, no. 1, pp. 450–461, 2024.

[5] L. Diez, A. M. Alba, W. Kellerer, and R. Agüero, "Flexible functional split and fronthaul delay: A queuing-based model," *IEEE Access*, vol. 9, pp. 151 049–151 066, 2021.

[6] M. Huang and X. Zhang, "Distributed MAC scheduling scheme for C-RAN with non-ideal fronthaul in 5G networks," in *2017 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 2017, pp. 1–6.

[7] J. Pérez-Romero, O. Sallent, A. Gelonch, X. Gelabert, B. Klaiqi, M. Kahn, and D. Campoy, "A tutorial on the characterisation and modelling of low layer functional splits for flexible radio access networks in 5g and beyond," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2791–2833, 2023.

[8] Small Cell Forum (SCF), "5g network fapi (nfapi) specification," Small Cell Forum (SCF), Tech. Rep. SCF225, 2020. [Online]. Available: https://www.smallcellforum.org/docs/5g-nfapi-specifications/

[9] ——, "5g fapi: Phy api specification," Small Cell Forum (SCF), Tech. Rep. SCF222, 2021.

[10] D. Villa, I. Khan, F. Kaltenberger, N. Hedberg, R. S. da Silva, S. Maxenti, L. Bonati, A. Kelkar, C. Dick, E. Baena, J. M. Jornet, T. Melodia, M. Polese, and D. Koutsonikolas, "X5G: An open, programmable, multi-vendor, end-to-end, private 5G O-RAN testbed with NVIDIA ARC and OpenAirInterface," *IEEE Transactions on Mobile Computing*, vol. 24, no. 11, pp. 11 305–11 322, 2025.

[11] S.-Q. Lee and M.-S. Lee, "A method of adaptive low-latency downlink transmission on fapi based 5g nr gnb," in *2022 13th International Conference on Information and Communication Technology Convergence (ICTC)*. IEEE, 2022.

[12] Y. Jia, Y. Zhong, M. Wang, J. Gao, P. Zhang, X. Liu, and X. Jin, "Aquifer: Transparent microsecond-scale scheduling for vRAN workloads," *IEEE Transactions on Services Computing*, vol. 17, no. 6, pp. 3171–3184, 2024.

[13] V. G. Douros, A. Garcia-Saavedra, and C.-P. Xavier, "nfapi: The standard interface for sdn-based 5g small cell networks," *IEEE Communications Standards Magazine*, vol. 2, no. 3, pp. 32–40, 2018.

[14] X. Wang *et al.*, "An open-source implementation of the 5g nr functional split option 6," in *2022 IEEE International Conference on Communications (ICC)*, 2022, pp. 1–6.

[15] V. Jacobson, "Congestion avoidance and control," in *Symposium proceedings on Communications architectures and protocols (SIGCOMM)*. ACM, 1988, pp. 314–329.

[16] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's retransmission timer," IETF RFC 6298, 2011.

[17] B. Ryu, R. Knopp, M. Elkadi, D. Kim, and A. Le, "5G-EMANE: Scalable open-source real-time 5G new radio network emulator with EMANE," in *2022 IEEE Military Communications Conference (MILCOM)*. IEEE, 2022.

[18] O-RAN Alliance, "O-ran architecture description," O-RAN Alliance Working Group 1, Tech. Rep. O-RAN.WG1.O-RAN-Architecture-Description, 2024.